ASSIGNMENT 7: Implement SVM Algorithm for Regression

1. Aim

To implement Support Vector Regression (SVR) using Python and predict continuous numerical values from sample data.

2. Objectives

1. To understand Support Vector Machines.
2. To study the regression application of SVM.
3. To understand the role of kernels in SVR.
4. To implement SVR using Scikit-learn.
5. To predict numerical outputs from input features.
6. To evaluate the model using regression metrics.

3. Theory

Support Vector Machine (SVM) is a supervised machine-learning technique primarily used for classification. Its regression version is known as Support Vector Regression (SVR).

SVR attempts to find a function that predicts numerical values while allowing a specified tolerance around the predictions. Errors within this tolerance do not receive the same penalty as errors outside it.

SVR can use different kernels to represent different relationships between inputs and outputs.

The most commonly used kernels include:

1. Linear kernel: Represents a linear relationship.
2. Polynomial kernel: Represents polynomial relationships.
3. RBF kernel: Can represent nonlinear relationships.

Three important parameters are:

1. C: Controls the penalty applied to errors outside the permitted tolerance.
2. Epsilon: Defines the tolerance margin around the predicted function.
3. Gamma: Controls how far the influence of individual training observations extends when using the RBF kernel.

Feature scaling is important because SVR can be sensitive to the numerical ranges of its input features.

In this assignment, sample data are divided into training and testing subsets. An SVR model using an RBF kernel is trained and evaluated using MAE, RMSE, and R-squared.

4. Algorithm

1. Start the program.
2. Import NumPy and the required Scikit-learn modules.
3. Create a sample numerical dataset.
4. Split the dataset into training and testing subsets.
5. Standardise the input features.

5. Explanation of the Code

1. `train_test_split()` divides the dataset into training and testing subsets.
2. `StandardScaler()` standardises the input features.
3. `make_pipeline()` combines scaling and regression into one workflow.
4. `SVR(kernel="rbf")` creates a Support Vector Regression model using the RBF kernel.
5. `C=100` sets the error penalty parameter.
6. `epsilon=2` defines the tolerance margin.
7. `model.fit()` trains the regression model.
8. `model.predict()` generates predictions for the test observations.
9. mean_absolute_error() calculates MAE.
10. np.sqrt(mean_squared_error()) calculates RMSE.
11. `r2_score()` calculates R-squared.

6. Applications

1. Predicting product prices.
2. Forecasting sales and demand.
3. Estimating industrial measurements.

4. Predicting environmental measurements.
5. Analysing nonlinear numerical relationships.
6. Forecasting values in scientific research.

7. Advantages

1. Can model nonlinear relationships using kernels.
2. Uses a tolerance margin to control prediction errors.
3. Can perform well on some moderate-sized datasets.
4. Offers flexibility through different kernel choices.
5. Does not require a straight-line relationship between inputs and outputs.

8. Limitations

1. Performance depends on selecting suitable parameters.
2. Feature scaling is generally necessary.
3. Training can be expensive for very large datasets.
4. Model interpretation is more difficult than simple Linear Regression.
5. SVR may perform poorly if the selected kernel does not match the data.

9. Conclusion

Support Vector Regression was implemented using an RBF kernel to predict continuous numerical values. The dataset was divided into training and testing subsets, and the model was evaluated using MAE, RMSE, and R-squared. The experiment demonstrated the application of SVM techniques to regression problems.